Portfolio

Initial release of the Hospital Patient Referral, Treatment and Administration System

Item	Detail
Module	UFCEP6-0-3 Advanced Information Systems Development
Assessment	Portfolio (50% of the unit) — initial release, ready for client review
School	School of Computing and Creative Technologies, UWE Bristol
Group	max, finn, henry, peter (four members; group size agreed with the tutor)
Submission	Blackboard, group submission (no 48-hour grace period)
Repository	https://github.com/XieQiBin0490/camunda-hospital-referral-group
Version submitted	tag p6-submission
Review snapshots	tag sr1, commit 95bf652 (increment at SR1, 17 September 2026); tag sr2, commit 772d928 (initial release with the 26 September acceptance evidence)
Demonstration	Demonstration of the working initial release at the scheduled session

Release identity used throughout this pack. The artefacts described are the initial release of the system (build 1.0), snapshotted as tag sr2: the four models, 68 Camunda Forms and six deployed process definitions, executed on 26 September 2026 (18 of 18 acceptance cases). One form, form-register-referral-landscape, was added after that snapshot with the S1 gateway fix recorded as DEF-19, so the repository at the submitted version contains 69 forms. Where a count appears, the text says whether it is the sr2 snapshot or the submitted version; neither is an error.

Table numbers run continuously across the whole portfolio, so a section document (one criterion on its own) starts at the number its section has reached. That is deliberate: the numbering is the same in the merged file and in the section files, so a reference to Table 20 means the same table wherever it is read.

Contents

Section	Deliverable	Criterion	Weight
1	Operational BPMN model	Operational BPMN (LO2, LO4)	15%
2	External workers	External workers (LO2, LO4, LO5)	15%
3	Camunda Forms	Camunda Forms (LO1, LO4, LO2, LO5)	15%
4	Project and test plan	Project & Test Plan (LO3, LO5)	10%
5	Marked standups and sprint reviews	SU1 5%, SU2 5%, SR1 10%, SR2 10%	30%
6	Demonstration	Demonstration (LO2, LO4, LO5)	15%
7	Evidence index and references	Supports every criterion above	—

How this portfolio is organised

One document per marking criterion, in the order of the marking scheme. Each document does three things: it states what was built, it points at the evidence that shows the thing works, and it states the limitations the group knows about. The rule applied throughout is that a claim without an evidence pointer is not made.

Evidence pointers use repository paths and, where a run exists, the process instance key the engine itself issued, so any result can be re-read in Operate rather than taken on trust. Section 7 is an index from marking criterion to the evidence that answers it.

Four kinds of evidence are kept separate, because they answer different questions: model validation (the file is well formed and lints clean), deployment (the engine accepted it), execution (instances reached the expected state), and inspection (a property is visible in the model or the code but was not executed). Where something was only inspected, it is labelled as such.

Members and responsibilities

Member	First-owner responsibilities	Second-owner support
max	Operational model, model integration, cross-pool message flows, deployment	requirements traceability, evidence files
finn	External workers, failure handling and the data store	forms, worker configuration
henry	Requirements, traceability, variable-to-gateway contract, documentation	model validation, evidence
peter	Camunda Forms, form validation, acceptance test harness and execution	deployment, worker tests

Every activity in the backlog carries a first owner and a second owner. The second owner keeps current enough to take the activity over; section 5 records where that handover was actually exercised.

1 Operational BPMN model

Criterion: Operational BPMN (15%). Learning outcomes LO2 and LO4.

The deliverable is an executable BPMN model of the hospital referral pathway, configured for Camunda 8 and deployed to a running engine, together with the evidence that its normal, alternative and exception paths execute.

1.1 What the model family contains

The pathway is modelled as four files: one merged operational model that runs, and three strategic views that place the same activities in their organisational context. The operational model is the deliverable for this criterion; the strategic views are submitted with the portfolio unit and are listed here only so the reader knows why four files exist.

Model	Process definition(s)	Pools (executable + black box)	Lanes	Flow nodes	Message flows
O1 Operational Merged	PR_Operational_Merged, PR_ReferringOrganisation	2 + 7	8	60	15
S1 Strategic Landscape	PR_Landscape	1 + 7	—	25	15
S2 Strategic Referral To Authorisation	PR_ReferralToAuthorisation	1 + 5	7	37	9
S3 Strategic Treatment To Aftercare	PR_TreatmentToAftercare, PR_EnquiryHandling, PR_ManagementReporting	3 + 6	11	79	13

Table 1. Model family. Node and flow counts are read from the .bpmn files as they stand in the submitted repository, which includes the two post-review additions described on the cover. The deployment and execution evidence in Table 6 is the 26 September initial-release run, when six process definitions and 68 forms were deployed.

1.2 Participants and collaboration

The case study requires collaboration between organisations that are independent of one another, so the model is a BPMN collaboration rather than a single process. The hospital is a white-box pool: its internal divisions are modelled as lanes because the case study attributes work to named teams. Every external party is a collapsed black-box pool, because the case study does not describe their internal behaviour and the hospital has no authority over it.

Participant	Type	Process definition
Specialist Hospital Service (Hospital Patient Administration System)	Executable white box	PR_Operational_Merged
Referring Organisation (GP or another hospital)	Executable white box	PR_ReferringOrganisation
Patient / Authorised Representative	Collapsed black box	—
External Scheduling Service	Collapsed black box	—
External Correspondence Service	Collapsed black box	—
Other Specialist Service	Collapsed black box	—
External Clinical Services (treatment, laboratory, imaging)	Collapsed black box	—
Funding Organisation / Insurer	Collapsed black box	—
External Payment Service Provider	Collapsed black box	—

Table 2. Participants of O1_Operational_Merged. Two pools are executable, so the collaboration runs rather than being drawn.

1.3 Responsibilities

The hospital pool declares eight lanes. Work is attributed to the team the case study makes accountable for it, and a lane exists only if it carries work — an empty lane asserts a responsibility the process does not discharge.

Hospital lane	Flow nodes in lane
Medical Secretaries	10
Consultant / Clinical Team	10
Outpatient Bookings Team	9
Call Handling Team	3
Treatment & Chemotherapy Bookings Team	7
Finance Team	12
Clinical Nurse Specialist Team	2
Patient Pathway Coordinators	1

Table 3. Lane attribution in PR_Operational_Merged.

Each user task also carries a BPMN documentation element stating the business rule or the responsibility that the task discharges, and a candidate group naming the team allowed to claim it. The documentation elements are what the requirements traceability traces against, and the candidate groups are what the Tasklist uses to separate the roles.

49 of the 99 user tasks in the four models carry such a documentation element; all user tasks carry a candidate group.

1.4 Paths that the model covers

The model is required to demonstrate normal, alternative and exception paths. The table lists the paths, the decision that selects each one, and the acceptance case that exercises it. Case identifiers are the ones in section 4.

Path	Selected by	Observable outcome	Case
Referral accepted, treatment schedule confirmed	referralDecision = accepted and every downstream gate passes	instance completes after T_ConfirmTreatmentSchedule	TC-13
Supporting documentation missing	documentsComplete = false	request sent, referral returns to the completeness check	TC-02
Referral rejected	referralDecision = rejected	outcome notification sent; no booking task reached	TC-03
Referral redirected	referralDecision = redirected	redirect notice sent; instance completes	TC-04
No suitable appointment slot	suitableSlotFound = false	escalated to the Patient Pathway Coordinator	TC-05
Appointment inside the 14-day window	appointmentWithin14Days = true	telephone task created for the Call Handling Team	TC-06
Appointment outside the 14-day window	appointmentWithin14Days = false	telephone task skipped	TC-07
Contact unsuccessful	contactSuccessful = false	attempt recorded, further attempt made	TC-08
Patient declines consent	consentGiven = false	pathway closes with T_AdviseNoTreatment	TC-09
Treatment request not authorised	requestAuthorised = false	returned to the clinical team, no capacity requested	TC-10
External capacity unavailable	treatmentCapacityConfirmed = false on the first attempt	booking held pending with a recorded retry	TC-11
Funding not approved	fundingApproved = false	funding approval requested and recorded	TC-12

Path	Selected by	Observable outcome	Case
Advance payment taken and confirmed	advancePaymentRequired = true, paymentStatus = confirmed	payment recorded, schedule confirmed	TC-14
Payment declined	paymentStatus = declined	failure handled, further attempt permitted, no duplicate booking	TC-15
Payment taken but not confirmed	paymentStatus = no response	marked for investigation; no second request	TC-16
Urgent treatment without confirmed payment	paymentStatus = urgent clinical need	authorised, reason recorded, referred to Finance	TC-17
No advance payment required	advancePaymentRequired = false	no payment task created	TC-18

Table 4. Paths, the data that selects them and the case that exercises them.

1.5 The data contract that makes the paths selectable

A branch is chosen by process data, never by editing the model. The table below reads the decisions out of the BPMN condition expressions themselves, so it can be checked against the files rather than trusted.

Gateway	Name	Outgoing conditions (as written in the model)
G_DocumentsComplete	All expected supporting documents available?	documentsComplete = false; documentsComplete = true
G_ReferralDecision	Referral decision?	referralDecision = "information required"; referralDecision = "rejected"; referralDecision = "redirected"; ref
G_UrgentReferral	Urgent referral requiring expedited booking?	referralUrgency = "urgent"; referralUrgency = "routine"
G_SlotsAvailable	Suitable appointment available within the requ	suitableSlotFound = false; suitableSlotFound = true
G_WithinFourteenDays	Appointment due within the next 14 days?	appointmentWithin14Days = false; appointmentWithin14Days = true
G_ContactSuccessful	Patient confirmed the appointment?	contactSuccessful = false; contactSuccessful = true
G_PatientConsent	Patient consents to proceed with treatment?	consentGiven = true; consentGiven = false
G_RequestAuthorised	Request completed and authorised by an authori	requestAuthorised = false; requestAuthorised = true
G_CapacityConfirmed	External treatment capacity confirmed?	treatmentCapacityConfirmed = false; treatmentCapacityConfirmed = true
G_FundingApproved	Funding approved or no payment required?	fundingApproved = false; fundingApproved = true
G_AdvancePaymentRequired	Advance payment required before the appointmen	(default); advancePaymentRequired = true
G_PaymentOutcome	Payment outcome returned by the payment servic	paymentStatus = "declined"; paymentStatus = "confirmed"; paymentStatus = "no response"; paymentStatus = "urgen

Table 5. Gateway conditions in PR_Operational_Merged. A blank expression on one branch is the default flow; every diverging exclusive gateway declares one.

1.6 Deployment configuration

Item	Configuration
Execution platform	Camunda 8 (Zeebe) 8.10 — declared in each .bpmn
Engine run	Camunda 8 Run 8.10.0-alpha5, single broker, RDBMS secondary storage
Modeler	Camunda Modeler 5.51.0
Form binding	zeebe:formDefinition with bindingType="latest" on every user task
Deployment unit	one deployment per model carrying its forms with it (09_Tests_and_Tools/deploy-all.mjs)
Job workers	twelve job types subscribed by the worker fleet (section 2)
Message correlation	correlation key referralReference on both directions of the cross-pool exchange

1.7 Validation, deployment and execution

Check	Tool	Result
Schema round-trip	bpmn-moddle 10.1.0 (Zeebe and modeler extensions)	0 warnings, four models
Camunda 8 compatibility lint	@camunda/linting 3.57.0 (Camunda 8 target)	0 reports, four models
Structural audit	validate-models.py: reachability, dead ends, gateway defaults, duplicate identifiers, empty lanes, lane membership	0 defects
Layout audit	nodes inside lanes, no overlapping shapes	0 defects
Deployment	Orchestration Cluster API v2 against 8.10.0-alpha5	6 of 6 definitions deployed
Execution	run-acceptance.mjs, 18 cases	18 passed, 0 failed, 0 incidents

Table 6. Validation evidence for the initial release: six definitions and 68 forms deployed, 18 of 18 cases executed on 26 September 2026. Logs: 05_Test_Evidence/validation.txt and 05_Test_Evidence/cluster.txt.

1.8 Limitations the group knows about

- The four models declare no bpmn:error definitions and carry no error boundary events, so a failure thrown as a BPMN error surfaces as an incident instead of being caught by the model. Business outcomes are handled through the outcome variables and the branches that already exist, which is why the three gating workers own their variables so carefully.
- All external participants are simulated by the worker fleet. No real scheduling, correspondence, payment or clinical provider is called, so the model is verified at the process boundary and not across a real integration.
- PR_ManagementReporting starts on a monthly timer, so it can be deployed but not created on demand; its service task is verified by the worker start-up coverage check rather than by execution.
- The strategic views (S1 to S3) are contextual documents for the portfolio unit. They are not part of the executable initial release and are not marked under this criterion.

The model is a collaboration because the referral does not start inside the hospital. A GP practice or another hospital composes it, and the case study gives the hospital no visibility of how that happens. Drawing one pool would have forced us to invent internal detail for the referring organisation, which is the one thing we know we do not know. The eight external participants are collapsed for the same reason: the patient, the scheduling service, the insurer, the payment provider and the external clinical services are black boxes because the hospital cannot see inside them and cannot tell them what to do. What the model can show is the dependency. Fifteen message flows, each one a thing the hospital needs from somebody it does not control.

Inside the hospital the pool is a white box with eight lanes. We used lanes rather than one unassigned process because the case study's central control is the separation of clinical, administrative and financial authority, and that separation is only visible when a task belongs to a named team. We also cut the lanes that carry no work. The Sprint 1 and 2 files declared lanes for the Finance Team, the Clinical Nurse Specialist Team, the Call Handling Team, Administrative Management and the Outpatient Bookings Team and then left them empty, which claims a responsibility the process never discharges.

The capacity and payment branches sit on the normal flow rather than on boundary events, and that was deliberate. A boundary event says "this is rare and exceptional". For this hospital, no suitable slot and a provider that does not answer are neither of those things: the hospital does not own the capacity, so it cannot guarantee that one exists. Putting the escalation to the Patient Pathway Coordinator on the main path, and the pending-and-retry as a loop rather than as an error handler, is the more honest picture of what the business does.

One thing had to be asserted rather than derived. The case study says urgent referrals must be expedited but never says what makes a referral urgent, so the gateway evaluates a variable a clinician sets. We record that as a limitation in section 1.8 rather than implying the rule is enforced, because a model cannot be more precise than the rule it encodes.

2 External workers

Criterion: External workers (15%). Learning outcomes LO2, LO4 and LO5.

The deliverable is the worker fleet that carries out every automated activity in the operational model, together with the evidence that it obtains work, uses the input variables, returns results, handles invalid input and service failure, and is tested.

2.1 The fleet at a glance

Item	Detail
Workers	13 job types: 12 service and send tasks plus 1 message-publish worker
Language and client	Java 17, Zeebe client, long polling against 127.0.0.1:26500
Service tasks in the models	22 service and send tasks, covering all 12 job types
Job types with no model binding	0 — the fleet refuses to start on a coverage gap
Unit tests	41, no engine and no network required
Engine scenarios	6, each driven only by completing user tasks
Job types executed against the engine	10 of 11; the eleventh is timer-bound (2.7)

2.2 Worker inventory

The three job types marked with a star own a variable that a gateway reads. That is the contract that matters: if one of them fails to write its variable the instance takes the wrong branch, or raises a FEEL null.

Job type	Model element(s)	Reads	Writes
request-missing-documentation	O1 T_RequestMissingDocuments; S1 T_RequestDocs; S2 T_RequestMissingDocs	patientId, referringOrganisation, missingDocuments	missingDocumentsRequested, missingDocumentList, documentRequestReference
send-referral-outcome	O1 T_NotifyReferralOutcome; S1 T_NotifyOutcome; S2 T_NotifyNotAccepted	patientId, referralDecision	outcomeNotified, outcomeLetterReference
redirect-referral	O1 T_RedirectReferral; S2 T_RedirectReferral	patientId, redirectDestination	redirectSent, redirectReference
query-available-appointment-slots ★	O1 T_QueryAvailableSlots; S2 T_QuerySlots	patientId, requestedSpeciality, referralUrgency, preferredTimeframeDays	suitableSlotFound, availableSlots, schedulingReference
send-appointment-confirmation-letter	O1 and S2 T_SendAppointmentLetter	patientId, appointmentReference, communicationPreference	appointmentLetterSent, appointmentLetterReference
request-treatment-capacity ★	O1 T_RequestTreatmentCapacity; S3 T_RequestTreatmentCapacity	patientId, treatmentCode, treatmentCycles, requiredResource	treatmentCapacityConfirmed, capacityReference, capacityAttemptCount, duplicateAppointmentCreated
request-secure-payment ★	O1 T_SendSecurePaymentRequest; S3 T_RequestSecurePayment	patientId, chargeAmount, currency, fundingApproved	paymentStatus, paymentReference, paymentAmount, cardDataStored
distribute-clinical-correspondence	S1 T_Correspondence; S3 T_DistributeClinicLetter	patientId, letterApproved, suspectedClinicalError	correspondenceSent, correspondenceReference
request-blood-test	S3 T_RequestBloodTest	patientId, cycleReference	bloodTestRequested, bloodTestReference, bloodTestDueDate
request-refund	S3 T_RequestRefund	patientId, refundAmount, refundAuthorised, paymentReference	refundStatus, refundReference
generate-management-reports	S3 T_GenerateManagementReports	reportPeriod	reportGenerated, reportReference, reportFigures

Job type	Model element(s)	Reads	Writes
referring-org-submit-patient-referral	O1 RO_ComposeReferral (referring organisation pool)	patientId, referringOrganisation	referralReference, referralStatus = "composed", referralComposedAt
referring-org-publish-patient-referral	O1 RO_SendReferral — message throw event	referralReference, patientId, referringOrganisation	referralStatus = "sent", referralMessageKey, referralSentAt

Table 7. One worker per job type the models declare. Source in 04_Java_Worker/src/main/java/uk/ac/uwe/hospital/tasks/.

2.3 How work is obtained and how execution continues

1. The worker subscribes to its job type with a long poll against the Zeebe gateway. The subscription is declared once, at start-up, and the fleet prints its coverage line before it begins polling.
2. A job carries the process variables as its input. The worker reads only the variables it declares, so a renamed variable fails at the contract test rather than silently taking a default.
3. The worker returns either a completion carrying output variables, a failure that the engine retries, or a BPMN error with a named code. Every worker returns through one place, AbstractWorker.onJob, so the fleet cannot disagree with itself about what a failure means.
4. The engine evaluates the next gateway against the variables the worker wrote, so the branch is selected by data. Execution continues without human intervention unless the next element is a user task.

2.4 Invalid input

A mandatory variable that is missing or malformed raises a ValidationException, which is reported as the BPMN error INVALID_JOB_INPUT naming the variable. The distinction is deliberate: a FEEL null on a gateway takes a branch nobody chose, so invalid data must fail loudly and must not be retried. JobContextTest (8 tests) pins this behaviour down by variable name.

2.5 Service failure

The fleet distinguishes four outcomes, because reporting one as another sends the patient down the wrong path:

Situation	Worker behaviour	Why
Provider declines (no slot, no capacity, payment declined)	writes the outcome variable and completes	these are business outcomes the model already branches on
Provider does not answer	TransientJobException — fail the job, one retry consumed	"no capacity" and "no answer" are different facts
Invalid input	ValidationException — BPMN error INVALID_JOB_INPUT	a data or form defect; it must be visible, not retried
Injected business error	BpmnErrorException with a named code	demonstrable failure handling
Anything unexpected	fail with no retries left — incident	an incident is the right signal for a defect a person must read

Table 8. Failure taxonomy, implemented in AbstractWorker.

2.6 The simulated services, stated as simulated

There is no real scheduling system, insurer, payment provider, laboratory or correspondence service behind the pools. Each worker simulates its provider deterministically: the outcome is a function of the input variables and, where a failure needs to be demonstrated, of an explicit injection flag. The simulation is therefore honest about what it proves — that the process behaves correctly for a given provider outcome — and silent about what it does not prove, which is the behaviour of any real provider, latency, or provider-side error formats.

The limitations that follow are recorded rather than hidden: response times are not modelled, no provider is authenticated, and a provider schema change would change the worker, not the model.

2.7 Testing

Two levels of evidence, because they catch different defects. Unit tests catch a worker that writes the wrong variable; the engine scenarios catch a worker that writes the right variable at the wrong time.

Suite	Tests	What it pins down
WorkerContractTest	16	one test per worker; the three gating workers hardest — that suitableSlotFound, treatmentCapacityConfirmed and paymentStatus really are written with the values the gateways test; that no card-shaped variable reaches the process; that a second payment of the same amount is suppressed; that the blood-test worker does not decide fitness
ExternalServicesTest	10	provider outcomes, and that a provider answering "no" is a business outcome while a provider not answering is a retry; that a forced decline applies to the first attempt only so the retry can succeed
JobContextTest	8	a missing or malformed variable is rejected by name
SupportTest	7	episode correlation, duplicate-payment detection, attempt counting, and that the audit log carries actor, time and action with no update or delete

Table 9. Unit tests: 41, run by mvn test with no engine and no network.

The engine scenarios are stronger evidence than the unit tests for integration, because run-worker-demo.mjs plays only the human roles: every service and send task must be completed by a running worker, so a scenario cannot pass unless the fleet is up and working.

Scenario	What it proves	Instance
happy	the workers drive the whole pathway; all three gating variables set	2251799813729686
capacity-unavailable	capacity refused on attempt 1, T_KeepPending reached, confirmed on attempt 2, no duplicate booking	2251799813729956
payment-declined	decline handled, further attempt succeeds, no card data reaches the process	2251799813730254
transient-retry	an injected provider failure is retried by the engine and recovered, not escalated	2251799813730539
aftercare-cycles	two treatment cycles, a clinic letter and a cancellation: request-blood-test, distribute-clinical-correspondence and request-refund, ending at E_PathwayReviewed	2251799813730803
enquiry	PR_EnquiryHandling runs to E_EnquiryClosed	2251799813731069

Table 10. Six worker-driven scenarios, all COMPLETED with no incident. Logs in 05_Test_Evidence/worker-run/workers-<scenario>.log.

The one job type not executed against the engine

generate-management-reports is bound only inside PR_ManagementReporting, whose only start event is a monthly timer (R/P1M). A process with no none start event cannot be created on demand — the API answers 409 Conflict — so there is nothing to subscribe to until the timer fires. It is covered by the start-up coverage check instead, and the attempt and its reason are recorded in run-worker-demo.mjs as scenario "reporting", marked skip. This is a stated limitation, not an omission.

2.8 Configuration management

- Configuration lives in 04_Java_Worker/src/main/resources/workers.properties with an environment override for every key, so no value has to be edited to run against a local cluster.
- The build is 04_Java_Worker/pom.xml; the delivered artefact is the jar built from the same revision as the sources in the repository.
- Every worker, the shared failure path and the store are first-owned by a named member and second-owned by another (section 5.9), so a worker cannot become undeliverable because one person is unavailable.

There is one worker per job type, thirteen of them, rather than one dispatcher that switches on the type. The job type is the only contract the engine and the code share, so putting the decision inside one class would move it somewhere the model cannot see. The fleet also prints its coverage line and refuses to start if a declared job

type has no subscriber, which turns a renamed job type into a start-up failure instead of a stuck instance. That check has already caught one rename.

Business outcomes are written to variables instead of being thrown as errors, and that follows from a gap in the models rather than from a preference. None of the four declares a bpmn:error or an error boundary event, so a thrown BPMN error has nothing to catch it and surfaces as an incident for an incident handler to read. The models already branch on suitableSlotFound, treatmentCapacityConfirmed and paymentStatus, so a worker that writes "no" lets the process take a branch somebody designed. We kept the distinction that matters: a provider that answers "no" is a business outcome and completes, a provider that does not answer is a transient failure and is retried. Treating those as one thing would send a patient to escalation because a network connection dropped.

Two things we would change. First, named error codes with boundary events on the service tasks, so the exception belongs to the model instead of to the worker. Second, an idempotency key on the payment request. The Finance Team's rationale shows why: sending the request serves the goal of collecting what is owed and works against the goal of never charging a patient twice, and the rule that a payment without a returned confirmation must be investigated rather than re-requested is a procedural answer to what is really a duplication risk. A key is the structural answer, and it is the first thing we would add.

3 Camunda Forms

Criterion: Camunda Forms (15%). Learning outcomes LO1, LO4, LO2 and LO5.

The deliverable is the form set that captures the information each user task needs and passes it to the process, together with the evidence that the bindings work, the inputs are validated, and the forms suit the people who use them.

3.1 Coverage

Item	Detail
User tasks in the four models	99
User tasks with a form	99 (100%)
Distinct forms	69
User tasks with a candidate group	99 (100%)
Distinct candidate groups	10
Form components	237 (168 inputs and 69 display elements)
Required inputs	157
Inputs with a constraint	70 pattern, 17 fixed-value lists

3.2 How the forms are bound

Every user task carries a zeebe:formDefinition with an explicit formId and bindingType="latest", and a zeebe:assignmentDefinition naming the candidate group. A form is not part of the .bpmn file, so the model and the forms it references are deployed together in one deployment by 09_Tests_and_Tools/deploy-all.mjs. Deploying the model alone leaves the first user task with a FORM_NOT_FOUND incident, which is exactly what happened before the forms were generated and deployed with the models; the defect is recorded in the repository defect log as DEP-04 and closed by that change.

Why bindingType is written explicitly: an earlier version of the attribute was misspelt as binding="deployment". The engine ignored an attribute it did not know and used its default, latest, so the Modeler Deploy button appeared to work while a teammate deploying their own copy of the forms silently replaced every drop-down with an old text box. The attribute is now correct, and the binding mode is switchable with 09_Tests_and_Tools/set-form-binding.ps1.

3.3 Control types and why each is used

Control	Count	Used for	Why this control
textfield	91	names, identifiers, references, short free text	single line; the only control that cannot accept a line break
text	69	headings and instructions inside a form	display only; keeps a long form readable without splitting the task
checkbox	39	yes/no decisions and confirmations	a decision that must be recorded even when the answer is no
select	17	controlled vocabularies — decision, urgency, funding route, outcome	prevents the value drift that made a gateway unselectable in an earlier increment
number	14	amounts, cycles, attempt counts	numeric entry, so a gateway comparing a number is never given a string
datetime	4	appointment dates and deadlines	a date the 14-day rule can be evaluated against
textarea	3	rationale, clinical notes, explanation	the reason for a decision has to be recordable in the clinician's own words

Table 11. Controls in use. An earlier increment used only text and textfield; the controlled vocabularies were added because gateways were comparing free text.

3.4 Validation

Validation is declared in the form itself, so it is enforced by the Tasklist before the task can be completed, and it is visible in the form file rather than hidden in the process.

Validation	Count	Where it matters
required	157	every variable a gateway reads, and every field the case study says must be recorded
pattern (regular expression)	70	identifiers such as the patient identifier and reference numbers
min/max length or value	35	amounts that must be positive, free text that must not be empty
fixed value list	17	decision, urgency, funding route, payment outcome — the values the gateways test

Table 12. Declared validation. Counts read from the .form files.

3.5 Variables exchanged with the process

The form key is the process variable. The table traces gating variables from the control that captures them to the gateway that reads them — the chain that has to hold or the model takes the wrong branch.

Variable	Captured by	Read by
suitableSlotFound	select in form-select-book-appointment (slot query result)	G_SlotsAvailable in PR_Operational_Merged
treatmentCapacityConfirmed	select in form-resolve-capacity-constraint	G_CapacityConfirmed
paymentStatus	select in form-record-payment and form-handle-payment-failure	G_PaymentOutcome and the urgent-treatment branch
referralDecision	select in form-record-referral-decision	G_ReferralDecision (accepted, rejected, information required, redirected)
consentGiven	checkbox in form-record-patient-consent	G_PatientConsent
appointmentWithin14Days	checkbox in form-record-pending-booking	the 14-day telephone rule

Table 13. Three of these six chains are re-checked by validate-contract.py (step 9 of verify-all) and by WorkerContractTest.

3.6 The intended user, accessibility and consistency

The users of these forms are not the public. They are nine hospital teams, each with a narrow job: a medical secretary checking documentation, a consultant recording a clinical decision, a finance officer recording a payment, a call handler classifying an enquiry. The design consequence is that a form is scoped to one team and one decision, and carries the vocabulary that team already uses.

The table states what the current set does and does not evidence against the accessibility expectations of LO1 (a design that recognises the user and is Disability Discrimination Act compliant).

Expectation	Status	Evidence or gap
Every input has a visible label	Evidenced	all 168 input components carry a label; checked across the 69 form files
Layout is a single logical reading and tab order	Evidenced	components are declared in the order the task is performed; no two-column layout
Controlled values rather than free text where the value is fixed	Evidenced	17 fixed-value selects; this is also what makes the gateways selectable
Required fields are marked and enforced	Evidenced	157 required inputs; enforcement is by the Tasklist, not by the process

Expectation	Status	Evidence or gap
Error messages identify the field	Partially evidenced	the Tasklist reports the failed constraint against the field, but no custom error text is authored, so messages are the platform default rather than task-specific guidance
Help text or description on each field	Not evidenced	0 of 237 components carry a description; the task name and the BPMN documentation element carry the instruction instead
Keyboard-only operation and screen-reader behaviour	Not evidenced	the forms use the Camunda form-js renderer, whose accessibility behaviour was not audited by this group; no assistive-technology test was run
Colour contrast and font size	Not evidenced	theme defaults were used; no contrast measurement was taken
Alternative formats for patient-facing communication	Not implemented	the case study requires accessible formats and translation for patients; the initial release captures no such preference

Table 14. Accessibility position, stated rather than assumed.

Consistency across the workflow

Consistency is the third thing the criterion asks for, and it is the one that can be measured rather than asserted. Two measurements across all 69 forms:

- There are 56 distinct variable keys. No key is captured by more than one control type: referralDecision is a select everywhere it appears, patientId is a textfield everywhere, an amount is a number everywhere. A value that the process compares exactly is never collected as free text.
- The 69 forms serve 99 user tasks, so 30 tasks reuse a form rather than defining a second one for the same decision. The most reused forms are bound to two tasks each, usually the operational task and its strategic counterpart, which is what keeps the two views talking about the same thing.

Where consistency is deliberately broken is the label wording, and it is worth being explicit about it because a reader will notice. Seven keys carry a different label depending on where in the pathway they are read: the same field is "resource required" when the booking team asks and "resource confirmed" when it comes back, and an amount is "amount to refund" before the decision and "amount refunded" after it. That is a choice for the person filling the form at that moment, at the cost of a slightly harder read across forms. If we were doing it again we would keep the per-task wording and add the fixed variable name as help text, which is the one place those two requirements do not pull in the same direction.

3.7 Testing

Check	Script	Result
Every form parses and every component is well formed	validate-forms.py	all 69 forms valid
Every user task binding resolves to a form that exists	validate-bindings.mjs	99 of 99 bound, 0 missing
No control that does not belong to the task	validate-form-controls.py	0 inappropriate controls
Every form renders in the renderer the Tasklist uses	render-forms.mjs (form-js)	all forms render
Every gateway variable has a form source	validate-contract.py	0 worker inputs with no form source
Forms exercised inside a running instance	run-acceptance.mjs	18 cases completed through the forms

Table 15. Form test evidence. The same checks run as steps 7, 8 and 9 of verify-all.ps1.

3.8 Appendix — form inventory

One row per form: the fields it carries, how many are required, and how many user tasks in the four models bind to it. A form bound to more than one task is the same decision reused in the strategic and operational views.

Form	Fields	Required	Pattern	Controls	Tasks bound

form-advise-no-treatment	2	1	1	text, textfield	2
form-answer-administrative-enquiry	2	1	1	text, textfield	1
form-answer-clinical-enquiry	2	1	1	text, textfield	1
form-answer-financial-enquiry	2	1	1	text, textfield	1
form-apply-treatment-change	3	2	1	checkbox, text, textfield	1
form-approve-clinic-letter	3	2	1	checkbox, text, textfield	1
form-arrange-follow-up	3	2	1	checkbox, text, textfield	1
form-authorise-treatment-funding	3	2	1	checkbox, text, textfield	1
form-authorise-urgent-treatment	2	1	1	text, textfield	2
form-book-new-patient-appointment	3	2	1	checkbox, text, textfield	1
form-check-referral-documents	5	3	1	checkbox, select, text, textarea, textfield	2
form-classify-enquiry	3	2	1	select, text, textfield	1
form-clinic-letter-admin-check	6	4	1	checkbox, select, text, textarea, textfield	1
form-clinical-review-referral	3	2	1	select, text, textfield	2
form-complete-pathway	2	1	1	text, textfield	2
form-confirm-appointment	4	2	1	checkbox, text, textfield	2
form-confirm-follow-up	2	1	1	text, textfield	1
form-confirm-treatment-schedule	2	1	1	text, textfield	2
form-deliver-treatment-cycle	6	3	1	checkbox, datetime, number, text, textfield	2
form-determine-funding-route	3	2	1	checkbox, text, textfield	2
form-escalate-correspondence	2	1	1	text, textfield	1
form-escalate-higher-management	2	1	1	text, textfield	1
form-escalate-to-manager	3	2	1	number, text, textfield	1
form-expedite-urgent-booking	3	2	1	checkbox, text, textfield	2
form-follow-up-review	3	2	1	number, text, textfield	1
form-forward-accepted-referral	3	2	1	select, text, textfield	2
form-handle-patient-enquiry	2	1	1	text, textfield	1
form-handle-payment-failure	3	2	1	select, text, textfield	2
form-highlight-follow-up	2	1	1	text, textfield	1
form-issue-letter-reminder	3	2	1	number, text, textfield	1
form-mark-payment-investigation	2	1	1	text, textfield	2
form-new-patient-consultation	3	2	1	checkbox, text, textfield	2
form-offer-another-appointment	2	1	1	text, textfield	1
form-prepare-accessible-format	3	1	1	select, text, textfield	1
form-prepare-clinic-letter	3	2	1	checkbox, text, textfield	1
form-provide-clinical-advice	2	1	1	text, textfield	1
form-provide-cns-contact-details	3	2	1	checkbox, text, textfield	1
form-receive-referral	6	5	1	checkbox, select, text, textfield	1
form-record-consultation-consent	3	2	1	checkbox, text, textfield	1
form-record-contact-outcome	3	2	1	checkbox, text, textfield	2
form-record-enquiry-outcome	2	1	1	text, textfield	1
form-record-funding-approval	8	6	1	checkbox, number, select, text, textarea, textfield	2
form-record-non-attendance	3	2	1	checkbox, text, textfield	1
form-record-patient-consent	3	2	1	checkbox, text, textfield	2

form-record-payment	6	5	1	datetime, number, select, text, textfield	2
form-record-pending-booking	5	3	1	checkbox, number, text, textfield	2
form-record-referral-decision	4	2	1	select, text, textfield	2
form-record-refund	5	4	1	datetime, number, text, textfield	1
form-refer-unconfirmed-payment	2	1	1	text, textfield	2
form-refund-decision	5	3	1	checkbox, number, select, text, textfield	1
form-register-referral	7	6	1	checkbox, select, text, textfield	1
form-register-referral-landscape	8	7	1	checkbox, select, text, textfield	1
form-request-follow-up	4	3	1	checkbox, number, text, textfield	1
form-request-funding-approval	3	2	1	checkbox, text, textfield	2
form-resolve-capacity-constraint	3	2	1	checkbox, text, textfield	2
form-resolve-funding-issue	3	2	1	checkbox, text, textfield	1
form-resolve-payment-transaction	2	1	1	text, textfield	2
form-return-unauthorised-request	3	2	1	checkbox, text, textfield	2
form-review-cycle-results	4	3	1	checkbox, text, textfield	1
form-review-financial-impact	3	2	1	checkbox, text, textfield	1
form-review-management-reports	3	2	2	text, textfield	1
form-select-book-appointment	7	5	1	checkbox, datetime, number, text, textfield	2
form-telephone-patient	3	2	1	checkbox, text, textfield	2
form-treatment-booking-request	8	7	1	checkbox, number, select, text, textfield	2
form-treatment-modification-request	3	2	1	checkbox, text, textfield	1
form-urgent-clinical-concern	2	1	1	text, textfield	1
form-urgent-postponement	3	2	1	checkbox, text, textfield	1
form-validate-booking-request	3	2	1	select, text, textfield	2
form-validate-treatment-request	5	4	1	checkbox, number, text, textfield	2

Table 16. All 69 forms. Controls listed alphabetically.

The forms were written before we had any reason to look at them as more than data capture. The review changed that in a specific way. When we audited the 68 forms against the gateways they feed, we found ten fields where a free-text box was collecting a value the process compares exactly. referralDecision was a textfield in front of a gateway that tests four literal strings. referralUrgency and enquiryType had the same problem, and two letter-age fields were checkboxes feeding numeric comparisons, which cannot be right. Those ten became selects or numbers, each one justified by the line of the model that reads it; 08_Repository_Documents/form-field-change-log.md lists all ten with the line reference.

The same audit added 41 controls for data the process needed and no form collected, and validation on 115 fields that already existed. The pattern behind those numbers is the same in each team. A medical secretary checking documentation needed a checklist per expected document rather than one free-text note. A consultant needed a controlled decision plus somewhere to write the reason. A finance officer needed the reference, the amount and the date, and nothing that resembles card data.

On accessibility our position is deliberately narrow. Every input has a visible label, the components are declared in the order the task is performed so the tab order follows the work, and the Tasklist enforces the required fields before a task can be completed. What we did not do is equally clear: no field carries a description or in-line help, no assistive technology was used to test any form, no contrast measurement was taken, and the accessible-format requirement for patients is out of scope in this release because no preference is captured. We would rather state that plainly than claim Disability Discrimination Act compliance we cannot evidence, and the next step we would take is a description on each field plus a screen-reader pass over the five longest forms.

4 Project and test plan

Criterion: Project & Test Plan (10%). Learning outcomes LO3 and LO5.

This section states how the project was planned and run, and then sets out the test plan and the evidence that the testing described in it was actually executed.

4.1 How the project was planned

The work was planned as a prioritised product backlog, cut into four sprints. Every item carries a first owner, a second owner, an estimate in story points and hours, its dependencies, its acceptance conditions and a status. The backlog workbook is the planning record; the sheets are listed below so a reader can go straight to the evidence rather than take the description on trust.

Workbook sheet	What it holds
Project Backlog	the prioritised product backlog (user story, workstream, priority, story points, sprint, first and second owner, dependencies, acceptance criteria, status, hours)
Sprint 1 Backlog … Sprint 4 Backlog	the selected items for each sprint with a daily plan, estimates, owners and status
Assessment Mapping	each marking criterion mapped to the backlog items that satisfy it and to the file that evidences it
Risks & Assumptions	the project risk register with likelihood, effect, response and owner
README	how the workbook is to be read and maintained

Table 17. Structure of 01_Agile_Planning/Hospital_Patient_System_Project_Backlog_按评价逐项修正最终版.xlsx.

4.2 Sprint plan and what each sprint was for

Sprint	Planned window	Items selected	Sum of estimates
Sprint 1	14 - 21 September 2026. Presentation milestone: SR1/SR2 evidence due by 28 September 2026. 冲刺 1 计划窗口：2026 年 9 月 14 日-21 日。演示汇报里程碑：SR1/SR2 evidence due by 28 September 2026（SR1/SR2 证据须在 2026 年 9 月 28 日前完成）。	17	128 h
Sprint 2	22 - 28 September 2026 | Capacity: 4 people × 5 days × 4 h = 80 h nominal; 128 h task plan (execution re-estimates). 冲刺 2 计划窗口：2026 年 9 月 22 日-28 日｜名容量：4 人×5 天×4 小时=80h；任务计划 128h（按行重新估算）。	18	128 h
Sprint 3	13 - 19 October 2026 | Capacity: 4 people × 5 days × 4 h = 80 h nominal; 116 h planned	25	154 h
Sprint 4	20 - 22 October 2026 | Capacity: 4 people × 3 days × 4 h = 48 h nominal; 80 h planned	24	80 h

Table 18. Sprint windows and estimates, read from the workbook. The sum is the Est. Hours column of the selected items, not a capacity statement.

The first two sprints are the ones marked for this assessment. Sprint 1 established the baseline — scope, backlog, repository, an initial BPMN increment and the first forms — and Sprint 2 integrated the operational model with the workers and the forms, which is the increment the second sprint review assesses.

Two marked activities fall in each sprint: one standup and one sprint review. Section 5 holds the records. The sprint reviews were marked against the increment available on the day, so later improvements are recorded in the backlog rather than written back into an earlier review.

4.3 Estimating, capacity and how the plan met reality

We planned 128 hours in each of the two marked sprints: 17 items in Sprint 1 and 18 in Sprint 2, summed from the Est. Hours column of the sprint backlogs. Nominal capacity was 80 hours per sprint on the assumption we used throughout the project, four people at four hours a day over five working days. Both sprints were therefore planned at around 160% of the hours that existed, and both were delivered by using the last two days of the sprint on integration rather than on new work. The plan was not wrong about what needed doing; it was wrong about how much of it four people could do in a week.

Two things were underestimated, and neither of them was the modelling. The first was the merge. Three files written at different times on two execution platforms had duplicate element identifiers, pools pointing at the wrong process, empty lanes and five gateways with no default flow. We had estimated the merge as "put them together"; it was really a rebuild, and the audit behind it produced sixteen defect categories before a single instance could run. The second was the distance between "the model deploys" and "the model runs". Three XSD sequence violations and the form-binding defect only appeared once an instance started, so a task we thought was finished at mid-sprint was not finished until the last day.

What we estimated well. Each form is small and bounded, so the form work was predictable once the inventory existed. The acceptance harness was written before the exception paths were specified, which made each new case cheap to add: eighteen cases cost far less than the first three, because the polling loop and the expectation model were already there. Documentation was estimated badly in both directions, first as nothing and then as everything.

If we planned again, the merge would be its own backlog item with its own owner and an estimate, the deployment rehearsal would sit at the middle of the sprint rather than at the end, and the forms would be generated from a frozen model instead of being written in parallel with a model that kept changing the task list underneath them.

4.4 Risks, dependencies and contingency

The risk register is the Risks & Assumptions sheet of the workbook. The table below reproduces its columns so the register can be read alongside this document, and the rows are reproduced from the sheet.

ID	Type	Risk	Chance	Effect	Response	Owner
R-01	Technical	It is hard to link Camunda with outside services, so the working model may be late	High	High	Build one small working flow early; make fake outside services (PB-26); max and finn work together on the link-up; tell the team about problems at standups	max/ finn
R-02	Technical	Payment and funding flows have many special cases (card declined, cancelled, double payment, messages fail)	Mediu m	High	Make a special-case table early (PB-20); write tests before code; save how we check problems in the model and tests	peter/ finn
R-03	Time	Two units (UFCEP4-0-3 and UFCEP6-0-3) run at the same time and the hand-in dates are close	High	High	One Backlog with shared milestones; put work together every week; always do Must tasks first	henry

ID	Type	Risk	Chance	Effect	Response	Owner
R-04	Time	SR1 is due on 28 Sep 2026, so we have little time for the full BPMN and the acceptance criteria	Medium	High	Keep Sprint 1 small; write first criteria early and make them better in Sprint 2; split tasks into small parts	henry/ max
R-05	People	A team member is away or late with their work	Medium	Medium	Every task has a first and second owner; tell the team about problems at standups; keep take-over notes on the task board	all
R-06	People	Some people do more work than others, so personal marks may not be fair	Medium	Medium	Keep first/second owners on every task, document the planned SP distribution, and retain GitLab evidence. First-owner SP is specialist-heavy (max 66, peter 42, henry 39, finn 26), while named second-owner coverage is 33/27/48/57 SP.	henry
R-07	Scope	Some requirements are not clear (for example, the hospital has not agreed how to decide if a question is urgent)	Medium	Medium	Ask the tutors at standups and reviews; write down what we assume (see A-05); keep a list of open questions	henry
R-08	Rules	We must keep private patient data safe and keep audit records	Medium	High	Set access rules, audit records and only keep the data we need; run the non-functional tests (PB-30); keep money data small	peter/ finn

Three risks materialised, and they were the three we had recorded at sprint planning, which is either good risk management or a sign that we recorded the wrong risks. The first was the state of the Sprint 1 and 2 fragments. We knew they had been written at different times, but not that two of them used Zeebe and the third Camunda 7 attributes, or that element identifiers had been reused with different meanings across files. The response was to rebuild rather than patch, and the consequence was that the merge ate the first half of Sprint 2 instead of two days of it.

The second was form binding. Our assumption was that a form is part of the model, because a user task names one and the Modeler shows it. It is not: the form is a separate artefact that has to be deployed, and the attribute that names it had been misspelt in a way the engine silently ignored, which is why a teammate deploying their own copy of the forms replaced every drop-down with an old text box without anything failing. The response was to deploy model and forms in one deployment and to check the binding in a test rather than by eye. The consequence was a day lost to FORM_NOT_FOUND on the first user task of every instance.

The third was having a working cluster at the review. The contingency was to rehearse the run the day before and to keep the log, so that if the cluster refused to start we could present the recorded run rather than the room. We used the contingency twice, and the reason the acceptance evidence in this portfolio is a recorded run rather than a live one is that the environment is reproducible only while the cluster is up.

One risk we chose not to take: the i* modelling sat outside the critical path by design, because it depends on nothing the operational model needs. That decision is why it could be done at the end without moving the release date.

4.5 Test plan

Objectives and scope

The test plan has one question behind it: does the initial release behave as the case study requires on the paths the client will see at the demonstration, and does it fail in a way a person can act on when it does not? Testing therefore covers four levels — unit, contract, integration and system acceptance — and explicitly excludes performance, load, security and penetration testing, for which no target was agreed.

Level	What it tests	Harness	Cases
Unit	worker logic: variables written, provider outcomes, invalid input, the store	JUnit, `mvn test`, no engine and no network	41
Contract	that the variable a worker writes is the variable a gateway reads	WorkerContractTest, validate-contract.py	16 + 1 script
Integration	the whole pathway driven by a real worker fleet; only human roles are played by the harness	run-worker-demo.mjs / run-worker-scenarios.ps1	6
System acceptance	the deployed process against expected states, elements and variables	run-acceptance.mjs over the Orchestration Cluster API	18
Form	parsing, bindings, control appropriateness, rendering	validate-forms.py, validate-bindings.mjs, validate-form-controls.py, render-forms.mjs	4 scripts

Table 19. Test levels. The full test plan, including preconditions, test data and pass conditions, is 05_Test_Evidence/Acceptance_Test_Plan.md.

Environment and test data

Item	Detail
Runtime	Camunda 8 Run (c8run) 8.10.0-alpha5, Windows x86_64
Secondary storage	RDBMS
API	Orchestration Cluster API v2, http://127.0.0.1:8080/v2/
Modeler	Camunda Modeler 5.51.0
Workers	the 13-job-type fleet polling 127.0.0.1:26500
Test data	synthetic patients only, identifiers of the form P-TC-nn; no real or identifiable patient data at any point
Isolation	each case starts its own instance from the start message Patient referral; cases do not share state

Entry and exit criteria

- Entry: the four models lint clean and deploy; the fleet is up and reports full job-type coverage; the forms are deployed with the models.
- Exit (per case): every expected element reached, no element expected to be skipped was reached, every occurrence count met, the final state matched, no incident raised, and every expected variable held the expected value.
- Exit (release): all 18 cases pass; 0 incidents; every failure investigated and either fixed at source or recorded as a limitation.

The 18 system acceptance cases

One row per case, with the data that selects the path and the process instance the engine issued on the recorded run. The instance key is the evidence: the trace can be reopened in Operate.

Case	Objective	Expected (elements reached / not reached / end)	Result	Instance key
TC-01	Complete referral documentation is accepted for clinical review	T_ClinicalReviewReferral, T_RecordReferralDecision	PASS	2251799813703877
TC-02	Missing documentation triggers a request and a return path	T_RequestMissingDocuments, T_ClinicalReviewReferral	PASS	2251799813703946
TC-03	Rejected referral closes with a notification	T_NotifyReferralOutcome | not T_ForwardAcceptedReferral | end COMPLETED	PASS	2251799813704124
TC-04	Redirected referral is sent to another specialist service	T_RedirectReferral | end COMPLETED	PASS	2251799813704195
TC-05	No suitable appointment slot is escalated to the pathway coordinator	T_ResolveCapacity, T_SelectAndBookSlot	PASS	2251799813704263
TC-06	Appointment within 14 days requires a telephone call	T_TelephonePatient, T_RecordContactAttempt	PASS	2251799813704437
TC-07	Appointment more than 14 days away skips the telephone call	T_NewPatientConsultation | not T_TelephonePatient	PASS	2251799813704612
TC-08	Unsuccessful contact is recorded and a further attempt is made	T_TelephonePatient	PASS	2251799813704782
TC-09	Patient refuses consent and the pathway closes	T_AdviseNoTreatment | not T_CreateTreatmentBookingRequest | end COMPLETED	PASS	2251799813704974
TC-10	Unauthorised treatment booking request is returned to the clinical team	T_ReturnUnauthorisedRequest	PASS	2251799813705089
TC-11	Unavailable external capacity leaves the booking pending and retries	T_KeepPending	PASS	2251799813705271
TC-12	Unapproved funding raises a funding approval request	T_RequestFundingApproval, T_RecordFundingApproval	PASS	2251799813705449
TC-13	Happy path: referral accepted and treatment schedule confirmed	T_ConfirmTreatmentSchedule | end COMPLETED	PASS	2251799813705624
TC-14	Advance payment confirmed and recorded	T_SendSecurePaymentRequest, T_RecordPayment | end COMPLETED	PASS	2251799813705784
TC-15	Declined payment is handled without creating a duplicate booking	T_HandlePaymentFailure, T_RecordPayment	PASS	2251799813705967
TC-16	Payment without a returned confirmation is investigated, not re-requested	T_MarkForInvestigation, T_ResolveTransaction | end COMPLETED	PASS	2251799813706153
TC-17	Urgent treatment proceeds without confirmed payment and is referred to Finance	T_AuthoriseUrgentTreatment, T_ReferUnconfirmedPayment | end COMPLETED	PASS	2251799813706334
TC-18	No advance payment required: no payment task is created	T_ConfirmTreatmentSchedule | not T_SendSecurePaymentRequest | end COMPLETED	PASS	2251799813706533

Table 20. Execution record. Run 2026-09-26T16:17:23Z: 18 executed, 18 passed, 0 failed, 0 incidents. Full per-step traces in 05_Test_Evidence/acceptance-run.json.

Coverage of the operational model

Across the 18 cases, 23 distinct model elements are asserted as reached, including every task that owns a gateway variable and every end event the operational model declares except the enquiry and reporting pools. Coverage is expressed as elements asserted and observed, not as a percentage of nodes: a node count would reward visiting a task without checking anything about it.

What was not executed, and the deviation record

Not executed	Reason	Where it is stated
PR_ManagementReporting end to end	its only start event is a monthly timer, so the process cannot be created on demand	section 2.7; scenario "reporting" marked skip
PR_EnquiryHandling under the acceptance harness	it is outside the referral-to-treatment scope of the 18 cases	driven instead by the worker scenario "enquiry" (instance 2251799813731069)
Performance, load, security, penetration, accessibility	no target agreed and no tooling in scope	section 3.6 and the limitations in each section
Real external providers	all providers are simulated by the fleet	section 2.6

Table 21. Deviations from the test plan, stated rather than omitted.

The four levels are in this order because each is cheaper to run than the one below it and catches a different class of defect. The unit tests need no engine and no network, so they run on every change and catch a worker that writes the wrong variable name, which no model review can see. The contract level checks the seam between the two: the variable a worker writes is the variable a gateway reads. The integration level is where the harness plays only the human roles, so a scenario cannot pass unless the fleet is running; that is the only level that proves the workers and the model agree at runtime. System acceptance sits on top and checks states, elements and variables against the case study.

Two choices are worth defending. The harness polls rather than subscribing to events, because the RDBMS-backed secondary storage is eventually consistent and a task completed through the API is not always visible to a search issued immediately afterwards. Polling is slower and uglier, and it does not produce false failures. Second, every case asserts variables as well as elements. Reaching a task proves the flow arrived; it does not prove the gateway read the value that was set. A FEEL null on a gateway takes a branch nobody chose, and an element-only assertion would call that a pass.

What the plan missed is the part we would write differently. The twenty original criteria were written as activities, not outcomes. "Payment failure is handled" cannot fail, because nothing in it says what would be observed if it were not handled. That is why the defects that surfaced on deployment were invisible to the plan: three XSD sequence violations and a form-binding defect are configuration and deployment faults, and the criteria did not ask whether the file could be deployed at all. The nine-step verify-all script exists because of that gap, and its first two steps are exactly what the original criteria omitted.

The plan also has a stated boundary. Nothing here tests performance, concurrency or recovery, because no target was agreed for any of them, and no provider was called, because every external service in this release is simulated. Those are limits of the plan rather than findings, and they are recorded in Table 21 rather than left for the reader to discover.

4.6 Configuration management

Configuration management is the practice the rubric asks about in every criterion, so it is recorded once here and referenced from the others.

Practice	How it is applied	Evidence
One shared repository	all models, forms, worker source, tools and evidence are in the group repository; nothing lives only on a laptop	repository URL; section 7
Version identified per submission and per review	an annotated tag marks the state reviewed at each sprint review	the three tags are named in section 4.6(a)
Change record	every model, form and worker defect is logged with the correction and the evidence that closed it	08_Repository_Documents/defect-log.md
Reversible changes	model and form defects were fixed at source and re-verified, so the log points at a reproducible measurement rather than a claim	DEF-01…DEF-21, DEP-01…DEP-04
Deployment is scripted, not manual	one deployment carries a model and the forms it references, so the two cannot drift apart	09_Tests_and_Tools/deploy-all.mjs

Practice	How it is applied	Evidence
One command re-runs every check	nine verification steps with a single summary	09_Tests_and_Tools/verify-all.ps1

Table 22. Configuration management practices in force.

4.6(a) Version identifiers

Three annotated tags identify the versions this portfolio refers to. Each is a signed point in the history that a reader can check with git show; none of them is a description of a tag that does not exist.

Tag	Commit	What it marks	Why it exists
sr1	95bf652	the first executable BPMN increment, 17 September 2026	the snapshot preserved for SR1: seven files — the repository skeleton, hospital-referral-final.bpmn and the modelling justification
sr2	772d928	the initial release, 29 September 2026	the snapshot preserved for SR2, and the state that carries the 26 September acceptance evidence: 318 files including 68 Camunda Forms
p6-submission	= `git show p6-submission`	the version submitted for this assessment	identifies the repository version behind the demonstration and this portfolio

Table 23. Version identifiers. The two review snapshots are the "version or snapshot for each review" the specification requires; the submission tag is the "identify the submitted repository version" the demonstration deliverable requires.

5 Marked standups and sprint reviews

Criterion: Marked standups (SU1 and SU2, 5% each, individual) and marked sprint reviews (SR1 and SR2, 10% each, group) — 30% together. Learning outcome LO3.

This section holds the records of the four marked activities. Each record states what was presented, the evidence used, what the group decided as a result, and what changed in the backlog. The records are written from the notes taken at the session, not reconstructed afterwards; where a figure in a record differs from the repository today, the difference is a post-review improvement and is recorded in the backlog.

5.1 When the four activities happened

Activity	Sprint	Timing as published	Assessed focus
Standup 1 (SU1)	Sprint 1	during Sprint 1, date announced by the tutors	individual evidence of progress; next actions, blockers, ownership
Sprint review 1 (SR1)	Sprint 1	end of Sprint 1, before 28 September 2026	initial executable BPMN increment, configuration management, clear project plan
Standup 2 (SU2)	Sprint 2	during Sprint 2, date announced by the tutors	progress, follow-up actions, handover; initial workers and forms
Sprint review 2 (SR2)	Sprint 2	end of Sprint 2, by 28 September 2026	integrated workflow increment: operational model, workers and forms together

Table 24. The four marked activities. A shared meeting may evidence both units; each unit marks its own criteria.

5.2 Definition of done

The group used one definition of done for every activity, agreed in Sprint 1 and applied unchanged since. An item is Done when all of the following hold:

5. it meets the acceptance conditions agreed at sprint planning;
6. a second team member has reviewed it;
7. it is in the shared repository, not only on the owner's machine;
8. it is linked to the evidence that demonstrates it;
9. the models parse with zero moddle warnings and lint with zero Camunda 8 reports;
10. exports and forms are regenerated from the same revision as the model they belong to;
11. the task record shows status, current progress, the next action and any handover.

5.3 Standup 1 (SU1) — individual update

Each member gives a concise update: what was completed, what is next, and what is blocking — with the artefact in hand. Attendance alone earns nothing, so each row names the artefact or task record the update was evidenced from.

Member	Completed since the last standup (with evidence)	Next actions	Blockers, dependencies, handover
max	First executable BPMN file, hospital-referral-final.bpmn, opened and linted in the Modeler (commit 1ca635c)	Start S1, the level 1 landscape, and the strategic-to-operational mapping	Nothing blocking. The case study does not say how urgency is decided, so the gateway will evaluate a variable we set by hand
finn	Job-type inventory read off the model: the automated steps that will need a subscription	Write the first two workers, the slot query and the treatment capacity request	The model still names service tasks by hand, so a rename would orphan a worker. Waiting on the task names being frozen
henry	Repository skeleton (ae99745) and the modelling justification (95bf652)	Close the requirement list and start the traceability table	The requirement list cannot close until the model boundary is agreed. max holds the model file and can take the traceability over

Member	Completed since the last standup (with evidence)	Next actions	Blockers, dependencies, handover
peter	Form inventory: which user tasks need a form and which controls each one needs	Build the first forms for the referral intake	No user task carries a form reference yet, so nothing can be bound until the model is updated. henry is second owner on the inventory

Nothing at the standup was blocking outright, so the two blockers that mattered were both dependencies rather than problems: the urgency rule and the missing form references. Neither could be settled inside the sprint. Urgency stayed with max to raise with the module team, the form references stayed with peter until the task list was frozen, and henry took the traceability work off max the same afternoon. That handover is the one recorded in the backlog, and it is the reason the traceability table was finished in Sprint 2 rather than in Sprint 1.

5.4 Standup 2 (SU2) — individual update

Standup 2 is assessed on follow-through as well as progress: the actions agreed at SU1 are expected to have moved, and any handover to the second owner is expected to be visible in the task records.

Member	Completed since SU1 (with evidence)	Next actions	Blockers, dependencies, handover
max	O1 merged into one file; participant names corrected and duplicate identifiers removed; the referring organisation pool now sends the referral and waits for the outcome on the same correlation key	Deploy all four models and hand the exports to henry	Three files were rejected on deployment over element order (DEP-01 to DEP-03). Fixed at source, but it cost a day
finn	Worker fleet up on the gateway: the slot, capacity and payment workers write the three variables the gateways read	Add the refund and correspondence workers, then the scenario runs	The payment provider is simulated, so a decline can only be shown by injection. peter is second owner on the scenarios
henry	Requirement list closed at 44 identifiers with the source sentence for each; the acceptance test plan written	Finish the traceability rows for the non-functional requirements	Four rows stay open because they are platform properties rather than activities. They will be recorded as not supported rather than modelled
peter	68 forms generated and bound to every user task; the deployment now carries the forms with the model	Run the acceptance cases and keep the evidence	FORM_NOT_FOUND on the first user task of every instance until the forms were deployed with the model. finn is second owner on the test evidence

The actions from SU1 moved. The form references were added and the forms bound, so peter's blocker closed on the first day; the task names were frozen, which closed finn's; henry closed the requirement list and took the four non-functional rows forward as recorded limitations rather than as open work. The urgency question did not move at all, and we stopped pretending it would: it was re-scoped as a dependency on a decision the hospital has not made, and it is the one item carried into the next increment with no action against it.

5.5 Sprint review 1 (SR1)

SR1 assessed the first increment against the sprint goal. The review was expected to show an executable BPMN increment, evidence of configuration management and a plan the client could believe.

We showed one file. `sr1` is seven files in total: the folder skeleton, hospital-referral-final.bpmn and the modelling justification that goes with it. The model opened and linted cleanly in the Modeler and deployed as a single definition, and that was the increment. There were no forms in the repository and no worker code, so nothing beyond the model itself could be demonstrated. The project plan existed as the product backlog plus the two sprint backlogs. Configuration management existed as commits and nothing else: no tags, which is the gap we have since closed by tagging `sr1` and `sr2` so that this review and the next one can be reopened exactly as they were.

Three decisions taken on the day. The pathway would be split across three strategic views and one merged operational model, rather than one large diagram, because a single readable view of both the boundary and the detail does not exist. The original acceptance criteria would be kept next to any revision of them, so that a later improvement could not quietly hide an earlier failure. And the pool-naming defect would be treated as a rebuild of the model family rather than a patch, because the same names had been reused with different meanings and patching one of them would have left the others.

Feedback at SR1 and the change it produced:

Point raised	Agreed response	Backlog change	Owner (1st / 2nd)
The pool names do not match the processes inside them, and two pools share a name (DEF-03, DEF-04)	One participant per organisation, each name matched to its own process	The rebuild of the three fragments became a single Sprint 2 item with an estimate	max / henry
The 14-day telephone rule is missing, and a 14-day timer the case study does not describe terminates the process (DEF-14, DEF-15)	Make the rule an explicit gateway, and record every attempt with its outcome	New item; delivered as T_TelephonePatient and T_RecordContactAttempt	peter / finn

5.6 Sprint review 2 (SR2)

SR2 assessed the integrated increment: the operational model with its workers and forms running together. This is the review whose evidence this portfolio relies on most heavily, because the acceptance run and the worker scenarios were presented at it.

By SR2 the increment was the whole release. `sr2` holds 318 files: four models, 68 Camunda Forms, thirty worker source files, and the acceptance evidence. We demonstrated the operational model from the start message through to a confirmed treatment schedule with the fleet running, then opened a second instance on the declined-payment path to show a failure branch rather than describing one. The run the evidence points at finished at 2026-09-26T16:17:23Z with eighteen cases passed and no incidents, and that is the run recorded in Table 20.

The sprint goal was an integrated increment: model, workers and forms running together with acceptance evidence. Two of the three were complete on the day. The forms were bound and deployed, but the enquiry and reporting processes had been deployed and never driven, and reporting cannot be driven at all because its only start event is a monthly timer. We also had no role-based access control and no persisted store at that point. Those are the review's first two items below, and they were delivered after the release rather than inside it, so they belong to the next increment and not to this record.

Feedback at SR2 and the change it produced:

Point raised	Agreed response	Backlog change	Owner (1st / 2nd)
1. Implement database integration to support data persistence	Put an embedded store behind the existing store classes so no worker has to be rewritten, and prove it by writing in one process and reading in another	New item, first in the increment after the review	finn / henry
2. Implement role-based access control	Enable authorization on the cluster, create the ten candidate groups and one user per group, then verify visibility rather than documenting the design	New item; needs a cluster restart to verify	peter / finn
3. Resolve the issues in the BPMN model so that it accurately represents the business process	Triage every defect in the fragments, fix at source, then re-validate all four models with the same toolchain	The rebuild item is enlarged; each defect gets an ID	max / henry
4. Implement appropriate form validation to prevent missing or invalid inputs	Required and format validation on every field a gateway reads, plus the fields the case study says must be recorded	New item	peter / henry

Point raised	Agreed response	Backlog change	Owner (1st / 2nd)
5. Use appropriate input field types for the data being collected	Audit every form against the gateway it feeds and replace free text with selects and numbers wherever the process compares a fixed value	New item; the audit feeds the change log	peter / finn

5.7 Retrospectives

A short retrospective was held after each review. The record states what went well, what did not, and the one or two improvements carried into the next sprint.

Sprint 1 retrospective

Went well: the abstraction level was agreed in writing before anyone drew anything, so the three files that appeared had the same boundary even though they disagreed about almost everything else. The second-owner rule held, and the one handover it was built for (traceability from max to henry) happened without a gap.

Did not go well: we planned 128 hours against 80 hours of capacity, which made the sprint look late from the first day even while it was going to plan. We also left the merge out of the backlog entirely, as if three files written separately would join themselves up.

Carried into Sprint 2: the model family structure agreed before more BPMN is written, and the merge added as a backlog item with its own estimate and owner.

Sprint 2 retrospective

Went well: deploying to a real cluster was the single most useful thing we did. It found three element-order violations and the form-binding defect that the schema round-trip, the lint engine and the Modeler had all passed. Writing the acceptance harness before the exception paths were specified also paid off, because each new case after the third was cheap.

Did not go well: integration again landed in the last two days, and the forms were finished last, so the first deployment of the release had nothing to bind to and every instance raised FORM_NOT_FOUND until it did. Documentation was treated as the work that happens after the work.

Carried into the next increment: a deployment rehearsal at mid-sprint rather than at the end, the model frozen before the forms are generated, and documentation given its own backlog items so it cannot be the thing that absorbs the slip.

5.8 Feedback action record

Every point raised at a review is traced to an agreed response, an owner, and the evidence that it was completed. Where an action was not implemented, the reason and the outstanding implication are recorded rather than the row being dropped.

ID	Point raised (review)	Agreed response	Owner (1st / 2nd)	Evidence of completion / reason not done
FB-01	SR2: implement database integration to support data persistence	embedded store behind the existing store classes; write in one process, read in a fresh JVM	finn / henry	Implemented in the next increment. H2 with four tables; persistence check 7 of 7 (05_Test_Evidence/persistence/), 41 unit tests still pass
FB-02	SR2: implement role-based access control	enable authorization, create the ten candidate groups and one user per group, verify visibility	peter / finn	Implemented in the next increment. task-visibility.json: 10 users, 10 groups, 0 failures, an ungranted user sees 0 tasks where the secretary sees 785
FB-03	SR2: resolve the issues in the BPMN model so that it accurately represents the business process	triage every defect, fix at source, re-validate all four models	max / henry	Implemented. DEF-17 to DEF-21 recorded and closed; validate-models.py clean; six definitions deploy

ID	Point raised (review)	Agreed response	Owner (1st / 2nd)	Evidence of completion / reason not done
FB-04	SR2: implement appropriate form validation to prevent missing or invalid inputs	required and format validation on every field a gateway reads	peter / henry	Implemented. 157 required inputs and 70 pattern rules across 69 forms; validate-form-controls.py reports 0 inappropriate controls
FB-05	SR2: use appropriate input field types for the data being collected	audit each form against the gateway it feeds; replace free text with selects and numbers	peter / finn	Implemented. 10 control types corrected, 41 controls added, 115 validations added (08_Repository_Documents/form-field-change-log.md)
FB-06	SR2: the urgency rules are still not defined	obtain a rule set from the module team and model the escalation on it	henry / finn	Not implemented. The hospital has not agreed a set of urgency rules and the module team confirmed none would be issued before hand-in. Implication: urgency stays a variable a clinician sets, REQ-11 and REQ-35 are traced as partly supported, and the rule set is the first Should in the next increment
FB-07	SR2: add automated regression tests for role-based access	automate the role and audit checks so a regression is caught rather than inspected	peter / finn	Not implemented. There was no capacity left once the deployment defects and the two portfolio evaluations were re-scoped. Implication: RBAC and audit immutability are evidenced by a one-off script and by inspection, not by a suite that runs on every change

Table 25. Feedback action record. FB-01 to FB-05 are the five points raised at the marked reviews and each is closed with evidence; FB-06 and FB-07 are the two that were not implemented, with the reason and the outstanding implication recorded rather than the row being dropped.

5.9 Contribution

The activity columns below record the share of each activity carried by each member and sum to 100% per column. The Total column is the split of the group mark, agreed at the start of Sprint 1 to be equal, which is a different measure.

Member	Runnable model	External workers	Camunda Forms	Tests and evidence	Total
max	45%	25%	15%	20%	25%
finn	10%	40%	10%	25%	25%
henry	15%	15%	20%	30%	25%
peter	30%	20%	55%	25%	25%

Table 26. Implementation-unit contribution. Evidence is the first- and second-owner fields in the backlog and the commit history; retained for tutor moderation.

6 Demonstration

Criterion: Demonstration (15%). Learning outcomes LO2, LO4 and LO5.

The demonstration shows the integrated initial release working: the operational process, the worker fleet executing the automated activities, the forms capturing and validating input, and the resulting outcomes in Operate and Tasklist. Each member explains their own contribution and answers questions about it.

6.1 What is being demonstrated, and which version

Item	Detail
Release	initial release, build 1.0
Repository version demonstrated	tag p6-submission
Process definition driven	PR_Operational_Merged (and PR_ReferringOrganisation for the message round trip)
Workers	the 13-job-type fleet, running locally against 127.0.0.1:26500
Forms	68 forms deployed with the models in one deployment
Scenarios	one end-to-end success path and one failure path at minimum; the full menu is 6.3
Evidence fallback	recorded run plus the logs in 05_Test_Evidence/ — see 6.7

6.2 Before the session

Run this checklist in order, before the room is watching. Every line is a step that has gone wrong at least once.

#	Action	What good looks like
1	Start the cluster (Camunda 8 Run) and confirm the API answers	GET /v2/topology returns the cluster
2	Clear previous instances so the demonstration is not confusing	no running instances in Operate
3	Deploy models and forms in one deployment	six definitions and 68 forms deployed; no FORM_NOT_FOUND later
4	Start the worker fleet and read the coverage line	"13 of 13 declared types have a subscriber"
5	Publish the start message Patient referral	an instance of PR_Operational_Merged appears in Operate
6	Open the Tasklist in a browser and sign in as the secretary	the referral task list is visible
7	Have the acceptance log open in a second window	the 18/18 summary is one scroll away if the live run misbehaves

Table 27. Pre-demonstration checklist.

6.3 Scenario menu

Six scenarios are available; the session needs the first two as a minimum — one success path and one failure path — and the rest are used to answer questions. Each is backed by a recorded instance.

#	Scenario	What it shows	Instance
1	Happy path (acceptance TC-13)	referral accepted through to a confirmed treatment schedule, workers completing every automated step	2251799813705624
2	External capacity unavailable (worker scenario)	the pending-and-retry branch: capacity refused, booking held, retry succeeds, no duplicate	2251799813729956
3	Missing documentation (TC-02)	the request-and-return loop inside the model	2251799813703946
4	Payment declined (worker scenario)	the decline path with a permitted retry and no second booking or charge	2251799813730254
5	Payment taken but unconfirmed (TC-16)	the investigate-don't-re-request rule	2251799813706153

#	Scenario	What it shows	Instance
6	Urgent treatment without payment (TC-17)	the authorise-and-refer-to-Finance exception	2251799813706334

Table 28. Demonstration scenarios and the instance that evidences each.

6.4 Run sheet

The order below keeps the story linear: one instance is followed from the start message to a confirmed schedule, with the workers visible in the log at each step, and then one failure path is opened from the start to show the branch that matters.

Step	Do	Point at
1	Publish Patient referral from the referring organisation pool	the new instance in Operate
2	Complete the documentation check with items missing	the request task created, then the return to the check
3	Complete the clinical review and record the decision	the branch taken by referralDecision
4	Watch the slot query complete without a human	the worker log line for query-available-appointment-slots
5	Book the appointment, then complete the telephone task	the form and the recorded attempt
6	Record consent and create the treatment booking request	the request and its authorisation
7	Show capacity refused, then confirmed on the retry	capacityAttemptCount and T_KeepPending
8	Record funding, take the advance payment, confirm the schedule	paymentStatus and the final state COMPLETED
9	Open a second instance and take the payment-declined path	the failure handling and the permitted retry

Table 29. Run sheet. Steps 1–8 are the success path; step 9 is the failure path.

6.5 Each member explains their own contribution

max. I built the operational model and the integration behind it. The decision Iwould defend is the cross-pool message round trip: the referral is published by the referring organisation and the outcome comes back on the same correlation key, which is what makes the collaboration execute rather than illustrate. I would change the way we merged the three Sprint 1 and 2 files. Rebuilding them was right, but we started it two weeks too late.

finn. I own the worker fleet and the way it fails. The decision that matters is separating a provider that answers "no" from a provider that does not answer: one is a business outcome the model branches on, the other is a transient failure the engine retries. I would add named error codes and boundary events next, so the exception belongs to the model rather than to the worker.

henry. I did the requirements and the traceability, and the variable-to-gateway contract that sits between the model and the code. The decision I would defend is tracing only the variables a gateway actually reads, which is why the contract checks found ten controls that were collecting the right value with the wrong control type. I would start the traceability earlier, before the model froze, so the boundary questions surfaced while they were still cheap to answer.

peter. I built the forms, and I ran the acceptance cases. The decision I would defend is that a case passes only if the variables hold the expected values, not just if the right task was reached, because a FEEL null takes a branch nobody chose. I would write the criteria as outcomes before writing any test, which is exactly what the review told us and what the revised criteria now do.

6.6 Likely questions, and where the answer is

Question	Answer from
Why is the model a collaboration rather than one pool?	section 1.2
What happens if a worker never writes its variable?	sections 2.3 and 2.5, WorkerContractTest

Question	Answer from
How do you know the branch was taken by data and not by editing the model?	section 1.5 (the conditions) and the acceptance cases, which set variables only
Which form field drives the 14-day rule?	section 3.5
How do you know the forms are bound to the right tasks?	validate-bindings.mjs, 99 of 99 (section 3.7)
What is simulated, and what does that not prove?	section 2.6
Which path is least tested, and why?	section 4.5, Table 21
What would you do first if you had another sprint?	the second-release backlog in the portfolio unit; and the error-boundary limitation in 1.8
How was the group mark divided, and what is the evidence?	section 5.9

Table 30. Question-to-evidence map. Answers are in this portfolio, not improvised.

6.7 If the live cluster fails

The demonstration must not depend on a cluster that will not start. The fallback is prepared before the session: a short screen recording of the same run sheet, plus the recorded trace of the instance being discussed, opened in Operate if the engine is available for reads. The fallback is stated here rather than improvised, because a confidently handled fallback reads better than a live failure.

7 Evidence index and references

7.1 Criterion to evidence

Criterion	Weight	Where it is argued	Where it is evidenced
Operational BPMN	15%	section 1	02_BPMN_Models/*.bpmn; 05_Test_Evidence/validation.txt; cluster.txt; Operate screenshots
External workers	15%	section 2	04_Java_Worker/ (source, pom, workers.properties, DEPLOYMENT.md); mvn test; 05_Test_Evidence/worker-run/
Camunda Forms	15%	section 3	03_Forms/*.form; validate-forms.py, validate-bindings.mjs, render-forms.mjs output; form render captures
Project & Test Plan	10%	section 4	01_Agile_Planning/Hospital_Patient_System_Project_Backlog_按价逐项修正最终版.xlsx; 05_Test_Evidence/Acceptance_Test_Plan.md; acceptance-run.json
Standups SU1, SU2	10%	section 5.3, 5.4	records above; backlog task records; commit history
Sprint reviews SR1, SR2	20%	section 5.5, 5.6	review records; tags per review; feedback action log
Demonstration	15%	section 6	live demonstration; recorded fallback; the instances in Table 28

Table 31. Every criterion, where it is argued, and the artefact that answers it.

7.2 Repository map

Folder	What it holds
02_BPMN_Models/	the four .bpmn models and readable PDF/PNG/SVG exports
03_Forms/	the Camunda Forms, one file per form
04_Java_Worker/	the worker fleet: source, pom.xml, workers.properties, run scripts, README, DEPLOYMENT
05_Test_Evidence/	acceptance plan, run output, screenshots, verification logs, worker scenarios
08_Repository_Documents/	model inventory, defect log, assumption register, run guide, integration and RBAC records
09_Tests_and_Tools/	the validation, deployment, acceptance and verification harness
01_Agile_Planning/	product backlog, sprint backlogs, work division, version snapshots

7.3 References

Camunda (2025) Camunda 8 documentation: Zeebe, BPMN, forms and the Orchestration Cluster API. Available at: https://docs.camunda.io/ (Accessed: 20 October 2026).

Object Management Group (2013) Business Process Model and Notation (BPMN), Version 2.0.2. OMG document formal/2013-12-09. Available at: https://www.omg.org/spec/BPMN/2.0.2/ (Accessed: 20 October 2026).

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide: the definitive guide to Scrum. Available at: https://scrumguides.org/ (Accessed: 20 October 2026).

UWE Bristol (2026) Case Study: Hospital Patient Referral, Treatment and Administration System. Unpublished module case study, UFCEP6-0-3 Advanced Information Systems Development. Bristol: University of the West of England.

W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: 20 October 2026).

The list is deliberately short and every entry is used in the text: the BPMN specification for the notation and the element ordering the deployment rejects, the Camunda documentation for the engine, forms and API this release is built on, the Scrum Guide for the agile practice recorded in section 5, the module case study for every requirement, and WCAG 2.2 for the accessibility position stated in section 3.6.